

LOIP — Loan Onboarding Intelligence Platform

Refined Build Plan (Problem-Statement-First)

> ****Key changes from v1:****

- > 1. Phase 0 explicitly incorporates MIDV-500, MIDV-2020, RVL-CDIP, FUNSD, DocVQA, and dedicated income-document generators
- > 2. Phases are rebalanced to de-risk the core DoD pipeline first (Identity → Income → Affordability → Risk Score)
- > 3. Traceability contract is defined upfront as a typed schema

Traceability Contract (Defined Once, Enforced Everywhere)

Every piece of evidence and every derived output carries this contract:

```
```python
schemas/evidence.py

class SourceLocation(BaseModel):
 document_id: str # UUID linking to stored document in MinIO
 document_type: str # "aadhaar", "pan", "salary_slip", "bank_statement", etc.
 page_number: int | None
 coordinates: dict | None # {"x0": y0, "x1": y1, "y0": y0, "y1": y1} in PDF/image
 space
 extraction_method: str # "paddleocr", "surya", "qwen2.5-vl", "donut",
 "human_entry"
 model_version: str # e.g. "paddleocr-4.0", "qwen2.5-vl-7b-v1"

class ExtractedField(BaseModel):
 field_name: str # e.g. "pan_number", "annual_income", "employer_name"
 raw_value: str
 normalized_value: str | None
 confidence: float # 0.0 - 1.0
 source: SourceLocation
 verified_by: list[str] = [] # list of verification model IDs that confirmed this value

class EvidenceChain(BaseModel):
 claim: str # e.g. "applicant_annual_income = 1,200,000"
 supporting: list[ExtractedField] # fields that support this claim
 contradicting: list[ExtractedField] # fields that contradict
 reconciled_value: str | float
 reconciliation_method: str # "source_trust_weighted", "majority_vote",
 "highest_confidence"
```

```
confidence: float
...
```

All domain outputs (identity, income, affordability, fraud, risk) are **\*\*wrappers\*\*** around evidence chains. Every figure traces through ``ExtractedField.source`` to a specific document and extraction event.

---

## ## Phase 0: Foundation + Data Assets (Weeks 1-2)

### ### Infrastructure

Docker Compose with:

- PostgreSQL (evidence repository, application data)
- MinIO (document storage, model artifacts)
- OpenSearch (full-text search, audit logs)
- Neo4j (identity graph)
- Kafka (async event pipeline between domains)
- Redis (rate limiting, session cache, Feast online store)
- MLflow (experiment tracking, model registry)
- Evidently AI (drift monitoring config)
- Prometheus + Grafana (metrics collection, dashboards)

### ### Data Assets

Asset   Source   Purpose
--------------------------

--- --- ---
-------------

MIDV-500   Public dataset — 500 images of mock passports, DLs, etc.   Realistic ID document images for OCR training/testing
-----------------------------------------------------------------------------------------------------------------------------

MIDV-2020   Extended MIDV with more doc types and variations   Additional ID doc variants, different lighting/angles
----------------------------------------------------------------------------------------------------------------------

RVL-CDIP   400K document images in 16 categories   Pre-training LayoutLMv3 for Indian document classification
---------------------------------------------------------------------------------------------------------------

FUNSD   199 form images with key-value annotations   Fine-tuning Donut for form understanding
-----------------------------------------------------------------------------------------------

DocVQA   50K document pages with QA pairs   Validation of Qwen2.5-VL extraction accuracy
------------------------------------------------------------------------------------------

### ### Generators to Build

Generator   Output   Key Variables
------------------------------------

--- --- ---
-------------

`generate_pan_card.py`   PAN card image + metadata   Name, PAN, DOB, father name, photo
-----------------------------------------------------------------------------------------

`generate\_aadhaar.py`	Aadhaar card image + metadata	Name, Aadhaar number, DOB, address, gender
`generate\_salary\_slip.py`	PDF salary slip + metadata	Company name, employee name, PAN, UAN, basic, HRA, allowances, deductions, net pay, month/year
`generate\_bank\_statement.py`	PDF bank statement + metadata	Bank name, account number, transactions (date, description, credit, debit, balance), period
`generate\_form16.py`	PDF Form 16 + metadata	Employer TAN, employee PAN, gross salary, deductions under 80C, 80D, tax payable
`generate\_itr.py`	PDF ITR + metadata	PAN, assessment year, gross total income, deductions, total income, tax payable

Each generator has a **\*\*tamper mode\*\***: produce variants with mismatched names, inconsistent income figures, forged fields — controlled by a `tamper\_type` parameter.

### ### Deliverables

- `docker-compose.yml` with all services
- Database migrations (Alembic)
- MinIO bucket policies (one bucket per doc type, evidence bucket, model bucket)
- Neo4j constraint definitions (uniqueness constraints on PAN, Aadhaar, Phone, Email)
- Kafka topic definitions (one topic per domain event)
- MIDV-500, MIDV-2020, RVL-CDIP, FUNSD, DocVQA downloaded and indexed in MinIO
- All generators implemented as CLI scripts in `scripts/generators/`
- Test harness that validates each generator output is OCR-parseable
- Cookie-cutter application case generator: `generate\_case.py --tamper-type income\_mismatch --output ./test\_cases/case\_001/`

---

## ## Phase 1: Core DoD Pipeline (Weeks 3-8)

**\*\*Goal:\*\*** End-to-end prototype that takes identity docs + income docs + application → produces the full DoD output. Face verification, liveness, graph fraud, review UI deferred.

### ### 1a — Minimal Document Intelligence (Weeks 3-4)

**\*\*Supported document types (narrow scope):\*\***

- PAN Card
- Aadhaar Card
- Salary Slip
- Bank Statement

**\*\*Classification (LayoutLMv3):\*\***

- Fine-tune on MIDV-500 + RVL-CDIP + custom generated docs
- 4 classes only (PAN, Aadhaar, Salary Slip, Bank Statement) + reject class
- Confidence threshold: classify only if > 0.85, else flag for review

**\*\*OCR Pipeline:\*\***

```
```python
def ocr_document(image: np.ndarray) -> tuple[OCRResult, OCRResult]:
    primary = PaddleOCR.extract(image)    # always runs
    secondary = SuryaOCR.extract(image) if primary.confidence < 0.9 else None #
validation layer
    if secondary and abs(primary.confidence - secondary.confidence) > 0.15:
        return ConflictResult(primary, secondary) # triggers verification flag
    return primary
```
```

**\*\*Extraction (Qwen2.5-VL via vLLM/Ollama):\*\***

- Prompt templates per document type
- Output conforms to `ExtractedField` schema
- Donut fallback on well-structured forms where Qwen2.5-VL confidence < 0.7

**\*\*Field Extraction Targets:\*\***

| Document | Fields |

|---|---|

| PAN Card | PAN number, full name, father's name, date of birth |

| Aadhaar Card | Aadhaar number, full name, date of birth, gender, address (street, city, state, pincode) |

| Salary Slip | Employer name, employee name, employee PAN, UAN, basic, HRA, allowances, deductions, gross pay, net pay, month, year |

| Bank Statement | Bank name, account number, account holder name, period start/end, transactions list, opening/closing balance |

### 1b — Identity Trust Lite (Weeks 3-4, parallel with 1a)

**\*\*Skip:\*\*** face verification, liveness, identity graph (deferred to Phase 2)

**\*\*Entity Matching (BGE-M3):\*\***

```
```python
def verify_identity(application: Application, extracted_docs:
list[ExtractedDocument]) -> IdentityVerificationResult:
    # Cross-document field matching
    checks = [
        match("pan_number", [extracted_docs["pan"], application]),
```

```

        match("full_name", [extracted_docs["pan"], extracted_docs["aadhaar"],
application]),
        match("date_of_birth", [extracted_docs["pan"], extracted_docs["aadhaar"],
application]),
        match("address", [extracted_docs["aadhaar"], application]),
        match("father_name", [extracted_docs["pan"], application]),
    ]
    # Each match produces: field_name, pair of values, similarity_score, source_docs
    # Mismatch if similarity < 0.85 (name) or < 0.95 (PAN) or < 0.90 (DOB)
    ...

```

****Output:****

```

```python
class IdentityVerificationResult(BaseModel):
 identity_confidence: float
 mismatches: list[EvidenceChain]
 tamper_flags: list[str]
 verified_fields: dict[str, EvidenceChain] # field_name → reconciled value with
evidence
```

```

1c — Income Intelligence (Weeks 5-6)

****Salary Detection:****

- Parse salary slip: all monetary fields plus employer name, employee PAN, month
- Parse bank statement: identify recurring monthly credits as salary candidates
- Cross-reference: employer name on salary slip vs. application; PAN on salary slip vs. PAN card

****Income Reconstruction:****

```

```
monthly_income_salary_slip = gross_pay from latest slip (or average of last 3)
monthly_income_bank = median of recurring salary credits in last 6 months
annual_income_salary_slip = monthly_income_salary_slip × 12
annual_income_bank = monthly_income_bank × 12
annual_income_form16 = gross_total_income from Form 16 (if present)
annual_income_itr = gross_total_income from ITR (if present)
```

```

```

Reconciliation: source_trust_weighted_average({
    (annual_income_form16, 0.85) if present,
    (annual_income_itr, 0.85) if present,
    (annual_income_bank, 0.75),
    (annual_income_salary_slip, 0.65),
})

```

...

****Anomaly Flags:****

- `salary\_slip\_income\_vs\_bank\_credits` — difference > 30%
- `employer\_name\_mismatch` — salary slip employer ≠ application employer
- `pan\_mismatch` — salary slip PAN ≠ PAN card PAN
- `missing\_mandatory\_fields` — salary slip missing basic, HRA, or gross pay
- `income\_below\_min\_wage` — verified income < statutory minimum
- `bank\_credit\_volatility` — CV of monthly credits > 0.5

****Model:**** XGBoost regressor that predicts "true income" from all extracted features; trained on synthetic data where true income is known.

1d — Affordability Intelligence (Weeks 5-6, parallel with 1c)

****Inputs:**** Verified income (Phase 1c), EMIs from application, bank statement transactions

****Metrics Computed from Bank Statement:****

- Average monthly balance (last 3/6 months)
- Monthly total credits (salary + other)
- Monthly total debits
- EMI debit count (recurring fixed-amount debits)
- Overdraft / insufficient funds event count
- Cash flow volatility (std dev of net monthly cash flow)

****Derived Metrics:****

- `DTI = min(total\_monthly\_obligations / verified\_monthly\_income, 1.0)`
- `FOIR = min(total\_monthly\_obligations / verified\_monthly\_income, 1.0)`
- `Disposable Income = verified\_monthly\_income - total\_monthly\_obligations - estimated\_monthly\_expenses`
- `Liquidity Score = min(avg\_monthly\_balance / avg\_monthly\_debits, 1.0)`
- `Financial Stress Score = number\_of\_overdrafts / total\_months`
- `Cashflow Stability = 1.0 - min(cv\_of\_monthly\_net\_cashflow, 1.0)`

****Model:**** LightGBM classifier (can afford vs. cannot afford); trained on synthetic repayment data with known default outcomes.

****Output:****

```python

```
class AffordabilityResult(BaseModel):
 verified_income: float
 income_confidence: float
 income_evidence: list[EvidenceChain]
 dti: float
```

```

 foir: float
 disposable_income: float
 liquidity_score: float
 cashflow_stability: float
 affordability_score: float
 affordability_confidence: float
 anomaly_flags: list[str]
 ...

```

### ### 1e — Risk Decisioning (Weeks 7-8)

**\*\*Inputs:\*\***

- Identity Score (Phase 1b)
- Income Score + Anomalies (Phase 1c)
- Affordability Score (Phase 1d)

**\*\*Decision Engine:\*\***

```

```python
def decide(identity: IdentityVerificationResult, income: IncomeResult,
           affordability: AffordabilityResult) -> OnboardingDecision:

    # Hard rules (non-negotiable)
    if identity.identity_confidence < 0.3:
        return REJECT("identity_low_confidence", identity.mismatches)
    if affordability.dti > 0.6:
        return REJECT("dti_exceeded", affordability.evidence_chains)
    if income.verified_income < loan_application.requested_amount * 0.05:
        return REJECT("income_insufficient", income.evidence_chains)

    # Review triggers
    review_flags = []
    if identity.identity_confidence < 0.6:
        review_flags.append("identity_moderate_confidence")
    if len(income.anomaly_flags) > 0:
        review_flags.append(f"income_anomalies:{','.join(income.anomaly_flags)}")
    if len(affordability.anomaly_flags) > 0:
        review_flags.append(f"affordability_anomalies:
{'',.join(affordability.anomaly_flags)}")

    if review_flags:
        return REVIEW(review_flags)

    # Soft scoring
    score = xgboost_ensemble.predict(identity.score, income.score,

```

```

affordability.score)
    if score >= 0.7:
        return APPROVE(score)
    elif score >= 0.4:
        return REVIEW(["borderline_score"])
    else:
        return REJECT(["low_score"])
...

```

****Reason Codes Catalog:****

```

- `identity_low_confidence`
- `identity_mismatch_name`
- `identity_mismatch_dob`
- `identity_mismatch_pan`
- `income_insufficient`
- `income_inflation`
- `income_mismatch_salary_vs_bank`
- `employer_name_mismatch`
- `dti_exceeded`
- `affordability_low`
- `cashflow_unstable`
- `borderline_score`
- `manual_review_required`

```

****Output:****

```

```python
class OnboardingDecision(BaseModel):
 risk_score: float
 decision: Literal["APPROVE", "REVIEW", "REJECT"]
 decision_confidence: float
 reason_codes: list[str]
 evidence_chains: dict[str, EvidenceChain]
 risk_factors: list[dict]
...

```

### ### Phase 1 Deliverables

- End-to-end FastAPI pipeline: `POST /onboard` → accepts files + application data → returns `OnboardingDecision`
- Every field in the response traces through `EvidenceChain` to source documents
- Synthetic data test suite that validates:
  - Clean data → APPROVE
  - Income mismatch → REJECT or REVIEW with correct reason code
  - Identity mismatch → REJECT with identity flags
  - Every output field has populated `source` chain



- CLI: `python -m loip.evaluate --case-dir ./test\_cases/case\_001/`

---

## ## Phase 2: Expand Depth (Weeks 9-12)

### ### Remaining Document Types

| Document | Add in Phase 2 |

|---|---|

| Passport | Classification + Qwen2.5-VL extraction (name, passport number, DOB, expiry, nationality) |

| Driving License | Classification + extraction (DL number, name, DOB, address, validity) |

| Form 16 | Classification + extraction (employer TAN, employee PAN, gross salary, deductions, tax) |

| ITR | Classification + extraction (PAN, assessment year, gross income, deductions, tax payable) |

| GST Returns | Classification + extraction (GSTIN, turnover, tax period, tax liability) |

Fine-tune LayoutLMv3 with all document classes.

### ### Full Identity Trust

#### \*\*Face Verification:\*\*

- ArcFace: face embedding extraction from ID photo and selfie
- InsightFace: secondary embedding extraction, fallback if ArcFace confidence low
- Similarity: cosine similarity between embeddings; match if  $> 0.6$

#### \*\*Liveness Detection (MiniFASNet):\*\*

- Anti-spoofing on selfie input
- Score  $< 0.5 \rightarrow$  spoof detected

#### \*\*Identity Graph (Neo4j):\*\*

- Nodes: Person, Phone, Email, PAN, Aadhaar, Device, Employer, Bank Account, Address
- Edges: `HAS\_PAN`, `HAS\_AADHAAR`, `HAS\_EMAIL`, `HAS\_PHONE`, `USES\_DEVICE`, `WORKS\_AT`, `HAS\_ACCOUNT`, `LIVES\_AT`
- Queries: find all applications sharing same phone/device/address  $\rightarrow$  synthetic identity detection

### ### Fraud Intelligence

#### \*\*Graph Fraud:\*\*

- GraphSAGE: node classification (fraudulent vs. legitimate application)

- Node2Vec: node embeddings for similarity queries
- Signals: shared device across applications, shared phone, shared address, shared employer with different names

#### **\*\*Behavioral Anomaly:\*\***

- Isolation Forest: detect outlier applications (unusual income patterns, document timing, etc.)
- AutoEncoder: reconstruction error on feature vectors → anomaly score
- One-Class SVM: boundary-based anomaly detection

#### **\*\*Fraud Rules:\*\***

- Same device used for > 3 applications in 24 hours
- Same phone linked to > 2 different PANs
- Employment at company that doesn't exist (cross-reference)
- Income significantly above industry + role benchmarks
- Document metadata suggests manipulation (Adobe Photoshop, creation date in future)

#### **### Expanded Generator Tamper Modes**

| Mode | Effect |

|---|---|

| `income\_inflation` | Salary slip gross inflated by 30-80% |

| `income\_deflation` | Bank statement credits hidden |

| `identity\_mismatch` | Name differs between PAN and Aadhaar |

| `synthetic\_identity` | PAN + Aadhaar + name belong to different real people |

| `employer\_fraud` | Employer name is a shell company |

| `document\_forgery` | Document created with editing software, metadata exposed |

---

#### **## Phase 3: Explainability & Human Review (Weeks 13-15)**

##### **### Explainability**

**\*\*SHAP:\*\*** Applied to every model (identity matching, income prediction, affordability, risk ensemble):

- Per-feature contribution to each score
- Stored alongside decision in OpenSearch audit log

**\*\*LIME:\*\*** Applied to document extraction:

- Which parts of the document contributed most to each extracted field
- Visualization: heatmap overlay on source document

**\*\*Qwen3-32B Reviewer Copilot (via vLLM/Ollama):\*\***

```
```python
prompt = f"""
```

Given the following loan application, identity verification, income analysis, affordability analysis, and fraud analysis, produce:

1. A 3-sentence summary of the application
2. The key reason for the decision
3. Any inconsistencies or red flags
4. Recommended questions for manual reviewer

```
Application: {application.summary()}
Identity: {identity.summary()}
Income: {income.summary()}
Affordability: {affordability.summary()}
Fraud: {fraud.summary()}
Decision: {decision.summary()}
"""
```
```

- Output appended to the decision record for reviewer consumption

```
Audit Trail:
```
```

Every inference → OpenSearch document:

```
{
  "application_id": "...",
  "timestamp": "...",
  "model": "income_xgboost_v1",
  "input_hash": "sha256:...",
  "output": { ... },
  "shap_values": { ... },
  "evidence_chain": { ... }
}
```
```

### ### Human Review UI

**Tech stack:** FastAPI + Jinja2 templates (deployable without separate frontend build) or Svelte SPA.

**Views:**

1. **Queue** — list of applications sorted by risk score (highest first) with decision, reason codes, timestamps
2. **Review Detail** — side-by-side evidence explorer:
  - Left pane: document viewer (PDF/image with page navigation)
  - Right pane: extracted fields with confidence, source location, verification status
3. **Decision Panel**

- Current system decision (APPROVE/REVIEW/REJECT)
- Override dropdown (APPROVE/REVIEW/REJECT + reason code)
- Notes text area
- Submit button

4. **Audit View** — full evidence chain for any figure, SHAP contribution breakdown, Qwen3 copilot narrative

**Workflow:**

...

Application → System Decision → Queue → Reviewer Opens → Reviews Evidence →  
 → Agrees → Logged as "reviewer\_agreed"  
 → Overrides → Logged with reason code + notes → Sent for feedback collection

...

---

## Phase 4: Enterprise Hardening (Weeks 16-18)

### MLOps

**MLflow:**

- Model registry: every trained model registered with version, stage (staging/production), metrics
- Experiment tracking: hyperparameters, datasets, metrics logged for every run
- Promotion workflow: model in staging validated on holdout → promoted to production

**Feast:**

- Feature views defined for: identity, income, affordability, fraud, risk
- Online store (Redis) for real-time serving
- Offline store (PostgreSQL) for batch training
- Feature serving API consumed by domain models

**Evidently AI:**

- Data drift: feature distributions compared between training and serving
- Model drift: prediction distribution shifts
- Target drift: actual loan outcomes vs. predicted risk (requires feedback loop)
- Alerts: drift beyond threshold triggers retraining pipeline

**Retraining Pipeline:**

- Trigger: scheduled (weekly) or drift alert
- Input: all reviewed applications with reviewer feedback + loan outcomes
- Output: updated model → registered in MLflow → staged → validated → promoted

### Operations

## **\*\*RBAC/ABAC:\*\***

- Roles: `admin`, `reviewer`, `senior\_reviewer`, `manager`, `api\_consumer`
- Permissions per endpoint + per resource type
- FastAPI dependency: `get\_current\_user()` → `check\_permission(role, resource)`

## **\*\*PII Protection (Microsoft Presidio):\*\***

- Automatic PII detection on all stored fields (PAN, Aadhaar, Phone, Email, Name, Address)
- Encryption at rest for PII fields (PostgreSQL column-level encryption)
- Masking in logs: Aadhaar: `\*\*\*\*\* 1234`, PAN: `ABCDE\*\*\*F`

## **\*\*Consent Management:\*\***

- Consent records stored per applicant: `application\_id`, `consent\_version`, `consented\_at`, `consent\_document\_hash`
- Pipeline checks consent before processing any document

## **\*\*Audit Logging Middleware:\*\***

- Every HTTP request logged: method, path, user, timestamp, request\_id, duration, status\_code
- Stored in OpenSearch with 1-year retention

## **\*\*Rate Limiting:\*\***

- Redis + slowapi: 100 requests/minute per API key, 10 requests/minute per upload endpoint

## **\*\*Health Checks:\*\***

- `/health` → overall status
- `/health/ready` → all dependencies (DB, MinIO, Kafka, Neo4j, model endpoints) reachable
- `/health/live` → service is alive

## **### Documentation & Hardening**

- **Setup guide:** `docs/SETUP.md` — prerequisites, docker-compose, environment variables, first run
- **API reference:** auto-generated from FastAPI (OpenAPI/Swagger) at `/docs`
- **Ops runbook:** `docs/RUNBOOK.md` — deployment, backup/restore, scaling, common issues
- **Security audit:** `pip-audit` / `bandit` / `safety` — run in CI
- **Load testing:** `locust` scenarios for `/onboard` endpoint target: 10 req/s with p95 < 5s
- **Performance benchmarks:** throughput per domain, p95 latency per model

---

## ## Directory Structure

...

loip/

- |—— docker-compose.yml
- |—— .env.example
- |—— pyproject.toml
- |—— schemas/
  - |—— \_\_init\_\_.py
  - |—— evidence.py           # Traceability contract
  - |—— identity.py
  - |—— income.py
  - |—— affordability.py
  - |—— fraud.py
  - |—— decision.py
- |—— scripts/
  - |—— generators/
    - |—— generate\_pan.py
    - |—— generate\_aadhaar.py
    - |—— generate\_salary\_slip.py
    - |—— generate\_bank\_statement.py
    - |—— generate\_form16.py
    - |—— generate\_itr.py
    - |—— generate\_passport.py
    - |—— generate\_dl.py
  - |—— download\_datasets.py
  - |—— generate\_case.py
  - |—— seed\_db.py
- |—— domains/
  - |—— evidence/           # Domain 1
  - |—— document\_intel/    # Domain 2
  - |—— identity\_trust/     # Domain 3
  - |—— truth\_reconciliation/ # Domain 4
  - |—— income\_intel/       # Domain 5
  - |—— affordability/     # Domain 6
  - |—— fraud/             # Domain 7
  - |—— risk\_decisioning/   # Domain 8
  - |—— explainability/    # Domain 9
  - |—— human\_review/       # Domain 10
  - |—— mlops/             # Domain 11
  - |—— enterprise\_ops/    # Domain 12
- |—— models/
  - |—— base.py            # Base inference wrapper
  - |—— paddleocr\_wrapper.py

- | |— surya\_wrapper.py
- | |— layoutlmv3\_wrapper.py
- | |— qwen2\_5\_vl\_wrapper.py
- | |— donut\_wrapper.py
- | |— bge\_m3\_wrapper.py
- | |— arcface\_wrapper.py
- | |— minifasnet\_wrapper.py
- | |— graphsage\_wrapper.py
- | |— xgboost\_wrapper.py
- | |— lightgbm\_wrapper.py
- |— pipelines/
  - | |— \_\_init\_\_.py
  - | |— base.py # Base pipeline with Kafka producer/consumer
  - | |— onboarding.py # POST /onboard orchestrator
- |— web/
  - | |— api.py # FastAPI app
  - | |— auth.py # RBAC/ABAC middleware
  - | |— routes/
    - | | |— onboard.py # POST /onboard
    - | | |— review.py # GET/POST /review/\*
    - | | |— evidence.py # GET /evidence/\*
    - | | |— audit.py # GET /audit/\*
    - | | |— admin.py # Admin endpoints
  - | |— templates/ # Jinja2 templates (Phase 3)
  - | |— static/ # CSS/JS (Phase 3)
- |— tests/
  - | |— generators/
    - | | |— test\_pan.py
    - | | |— test\_aadhaar.py
    - | | |— test\_salary\_slip.py
    - | | |— test\_bank\_statement.py
  - | |— pipeline/
    - | | |— test\_onboarding.py
    - | | |— test\_identity.py
    - | | |— test\_income.py
    - | | |— test\_affordability.py
    - | | |— test\_decision.py
  - | |— domains/
    - | | |— ...
  - | |— fixtures/
    - | | |— clean\_case/ # All docs consistent
    - | | |— income\_mismatch/ # Salary slip vs bank statement mismatch
    - | | |— identity\_mismatch/ # PAN vs Aadhaar mismatch
    - | | |— income\_inflated/ # Salary slip inflated
- |— docs/

```
|—— SETUP.md
|—— API.md
|—— RUNBOOK.md
...
```

---

## ## Dependency Graph

...

Phase 0: Foundation + Data Assets + Generators (Weeks 1-2)

|



Phase 1: Core DoD Pipeline (Weeks 3-8)

- |—— 1a Document Intel (PAN, Aadhaar, Salary Slip, Bank Statement)
- |—— 1b Identity Trust Lite (no face/liveness)
- |—— 1c Income Intelligence
- |—— 1d Affordability Intelligence
- |—— 1e Risk Decisioning

|

▼ ◀—— DoD achieved end-to-end

|

Phase 2: Expand Depth (Weeks 9-12)

- |—— All document types (Passport, DL, Form 16, ITR, GST)
- |—— Full identity (face, liveness, graph)
- |—— Fraud Intelligence (graph fraud, anomaly detection)

|



Phase 3: Explainability + Human Review (Weeks 13-15)

|



Phase 4: Enterprise Hardening (Weeks 16-18)

...

**\*\*Key insight:\*\*** After Phase 1 (week 8), you have a working system that fulfills the **\*\*Definition of Done\*\*** — cross-verified identity, reconstructed income, affordability, risk score, approve/review/reject, every figure traced to source. Everything after that is depth and polish.

---

## ## DoD Validation Checklist (Test Suite)

After Phase 1, the test suite must pass:



- [ ] `test\_clean\_application\_approves` — matching docs, consistent income → APPROVE
- [ ] `test\_identity\_mismatch\_rejects` — PAN name ≠ Aadhaar name → REJECT with `identity\_mismatch`
- [ ] `test\_income\_inflation\_rejects` — salary slip says ₹12L, bank credits say ₹6L → REJECT with `income\_inflation`
- [ ] `test\_dti\_exceeds\_threshold\_rejects` — DTI > 0.6 → REJECT with `dti\_exceeded`
- [ ] `test\_low\_confidence\_reviews` — slight mismatches → REVIEW
- [ ] `test\_every\_field\_has\_source` — all output fields have populated `EvidenceChain`
- [ ] `test\_source\_chains\_trace\_to\_documents` — each claim's evidence chain links to a real MinIO object
- [ ] `test\_synthetic\_tampered\_doc\_flagged` — tampered salary slip → anomaly flag raised

---

## ## Key Design Decisions

1. **Async event-driven architecture** — Kafka between domains for loose coupling; each domain publishes and subscribes to typed events.
2. **FastAPI everywhere** — Unified framework across all HTTP APIs for consistency.
3. **Model serving** — Qwen2.5-VL and Qwen3-32B via vLLM/Ollama on-prem; smaller models loaded directly via ONNX/torch.
4. **Synthetic data strategy** — MIDV-500/2020 for ID images, custom PDF generators for income docs, tamper modes for all fraud categories.
5. **Domain-first, not microservices-first** — Logical domains within a single codebase initially; extract to microservices only when needed.
6. **Rules before models** — Hard business rules gate decisions before ML scores are applied; ensures auditability and regulatory compliance.
7. **Traceability by construction** — `EvidenceChain` is not a post-hoc addition; it is the core data structure that all domains produce and consume.